# Screening Tasks for Agentic Automation

A companion to the Agentic System Design Process: what to check between Phase 1 and Phase 2

J. A. Arroyo

Version 0.9.3 — 2 November 2026

**Abstract**

This document gives a first screen for deciding, before building anything, whether a task is worth testing with a language model and, separately, whether to test an agent. The explanation is short: look for an expert who spends their time carrying results between tools that already work, where there is evidence to judge the result against; then ask whether a fixed procedure, with its branches and retries, can do the job. Each point is illustrated with systems from 22 selected papers and preprints, listed in the appendix: nine from one engineering and materials-science group and thirteen from other groups. **The screen is my inference from recurring cases in those papers. None of them tests it as a predictive rule.** [inferred] marks a statement the papers do not themselves make.

**One set of three documents**

1. **Learning to Build Agentic Systems** (the guide): the courses, lectures and assignments behind the practices. *Use it to learn a topic.*
2. **Agentic System Design Process**: nine phases from framing a problem to running it in production. *Use it to design a system.*
3. **Screening Tasks for Agentic Automation**: a one-page worksheet, filled in after Phase 1 of the design process, that feeds Phase 2 and sets up the comparison run in Phase 7. *Use it to decide whether a task is worth it.*

This is document 3. Phase numbers and the four options (deterministic code, single LLM call, fixed workflow, bounded agent) are those of the design process[D]; the courses behind it are indexed in the guide[G].

## 1 The idea

Much expert work with software has the same shape. The expert runs a tool, reads what came out, decides what to try next, prepares the input for the next tool, and repeats until a check says the result is good enough. The tools do the computing. The person is the connection between them (Figure 1).

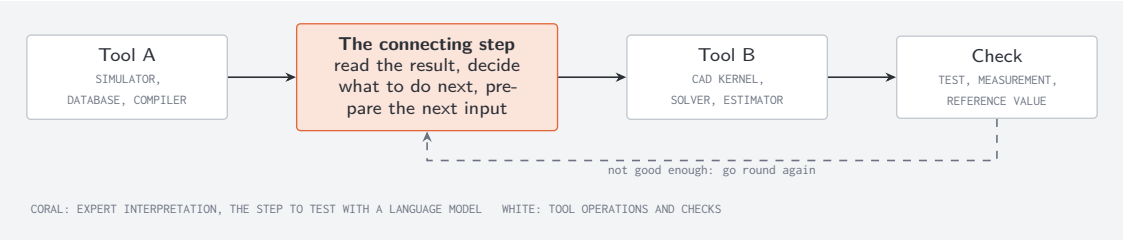



Figure 1: A recurring pattern: expert interpretation connects tool operations and checks. The tools must be usable and the checks tested; some checks stay with a person. The arrangement is my synthesis [inferred].

The connecting step is worth testing with a language model when three things are true:

1. **There is a connecting step.** Someone has to interpret a result before the next tool can run, and that interpretation is hard to write as rules.

2. **The tools can run.** Software does the calculating and simulating. The model may still have to form hypotheses, choose assumptions and interpret results, and those judgments need checking too.

3. **Something checks the result.** A test, a measurement, a reference value or a calibrated expert review says whether a step worked.

That is the whole of the first question. Passing it justifies a trial, not a prediction of success, and it says nothing yet about agents. The second, separate question is: **can a fixed procedure, with its branches and retries, do the job?** If it can, the system is a fixed workflow, the model fills in the connecting step at set points, and that is what to test first. If important decisions are left unresolved, compare it with a version in which the model chooses the next action from the results and decides when to stop or escalate. That component is an agent, and whether it earns its extra cost is something to test, not assume.

**Example: a rocket, and a software bug**

**RocketSmith**[2] designs high-powered rockets. A flight simulator, a CAD kernel and a slicer already exist; an engineer normally iterates between them, updating mass estimates and checking that the design still meets its stability target. The paper names its motivation as "friction between the various software tools" and notes that "delegating these numerically precise tasks to tools would result in accurate responses"[2, pp. 2, 6]. The connecting step is the engineer; the checks are simulated stability, slicer mass and, in the end, four flights.
**TDFlow**[8] repairs software defects. The tools are the repository and its tests; the connecting step is reading the code and writing a patch; the check is whether the tests pass. Its order of steps is written in advance: the system "prevents the LLM from deciding future actions"[8, p. 3]. The outer procedure is fixed, with small tool-using agents inside its stages. Given human-written tests, it reports 88.8% success on 278 attempted SWE-bench Lite cases[8, pp. 5, 11].

The two examples have the same shape, and both mix a fixed procedure with parts where the model chooses. TDFlow states its fixed outer workflow outright; RocketSmith also prescribes a sequence of skills[2, p. 19]. So ask the second question about one component at a time. The rest of this document turns each question into things you can check from existing work.

**What this replaces.** A natural first guess is that the good opportunities are tasks where rules written in code hit a hard wall. The papers do not establish that as a way to choose an agent. In the systems I read, software usually does the calculation, the simulation and the check, and a model does the interpreting, generating or coordinating. That is how the authors divided the work. It does not show that rules or conventional methods could not do the model's share. And a limit of rules does not by itself call for an agent, because a single model call and a fixed workflow get past rules too.

## 2 Scope: jobs that end, and services that keep watch

Before screening a task, say how the system would run. There are two cases [inferred]:

- **On demand.** A person or another program hands the system a piece of work. It runs, and it ends when that work ends.
- **Always on.** The system stays responsible for *noticing* that work is needed as events arrive, and may start a bounded run when one does.

This is an engineering distinction that I define here; I found no source that establishes it as a taxonomy. The closest is Lu et al.[25], who contrast agents that need an explicit instruction with an agent that predicts a useful task from observed events. Their agent either proposes help or stays silent, and a proposed task runs only after it is accepted[25, §§3.1–3.5].

Four other questions are often folded into this one. They are separate, and each needs its own answer:

- *How the world behaves.* Whether one decision affects later ones (episodic or sequential) and whether the world changes while the system deliberates (static or dynamic) describe the task environment, not what starts the program. An environment can be both episodic and dynamic[23, §2.3].

- *Reacting and taking initiative.* Responding promptly to changes in the environment and taking initiative in pursuit of goals are properties one agent can have together, not two kinds of system. Taking initiative does not require that nobody asked for the goal[24, "A Weak Notion of Agency"].
- *Memory.* Keeping information between runs does not mean running without pause. A system can learn across separate jobs[26, §4.1].
- *Who approves.* How much the system may decide and do without a person is set on a scale, and separately for gathering information, analysing it, choosing an action and carrying it out[27, §III, Table I]. An on-demand system can act without approval, and an always-on one can be required to ask.

Always on also need not mean that a language model runs all the time. Ordinary code can do the watching and call a model only when something needs interpreting [inferred].

**Example: a monitor that starts a job (an illustration, not from the 22 papers)**

A service watches a stream of readings with fixed thresholds. When a reading drifts out of range, it starts a bounded investigation that gathers evidence, tests explanations and drafts a response. If the response exceeds what the system is authorized to do, it waits for a person. The watching is code; the investigation is a job that ends; the approval rule is a separate decision.

**What this means for the screen.** The evaluations reported in the 22 papers concern bounded jobs: a paper, a design brief, a defect or a dataset is handed over and the job ends. None evaluates a service on deciding when a new job should begin [inferred]. That does not settle how every system is deployed; the laser-sintering system also describes process monitoring and overrides during a build[5, pp. 8–12]. The two questions below are built from those bounded jobs. For an always-on candidate, screen both the bounded job and the task of noticing when it is needed, starting the second from the simplest option: a threshold, a rule or a conventional model. This set of papers does not show when the noticing itself benefits from a language model.

# 3 First question: is the task worth automating with a language model?

Start with the simpler answers: could a script, a conventional statistical or machine-learning model, or a numerical optimizer meet the target? Could one model call with the right context do it? This follows Phase 2 of the design process, which chooses the simplest adequate solution and, checks conventional models and optimizers before the ladder. If an option meets the target, use it. If you do not know, run the smallest test that would tell you.

**Example: when simple search wins**

**Pepti-Agent**[3] refines peptide sequences against predicted properties. On 24 matched starting sequences that were already feasible, a scripted exhaustive search over single substitutions scored higher than the paper's conservative agent every time, on a score defined by the predictors. Compute and edit depth were not matched, and the manuscript says it "does not claim" to outperform simpler search[3, pp. 19, 22, 24].

Then check the three conditions.

**A1. There is a connecting step.** Take three to five finished pieces of real work and mark each point where a person had to interpret something before software could proceed: a paper, an error message, a geometric feature, a clinical passage. Expect most of the surrounding work to be mechanical.

**A2. The operations can run under your constraints.** List the operations on each side of the connecting step and how each is reached, including any wrapper, screen interaction or human handoff. Record licence, cost, data access and delay, and which actions the model would be allowed to take. Some physical steps stay with people: RocketSmith leaves fabrication, assembly and flight-readiness work to them, and the laser-sintering study[5] leaves handling and physical testing to them alongside its machine-control tools[2, pp. 19–21, 27, 31; 5, pp. 11–13].

**A3. Something checks the result.** Name two things, which may differ: the feedback available *while the system runs*, and the *final acceptance check*. For each, say what kind it is: a *reference* (a test, a measurement, a published value), a *proxy* (a learned predictor), or a *judgment* (a model judge or a human reviewer). Record what it covers, what it costs, how long it takes and who owns it; try it on a few results you already know to be good and bad;

and make sure the final check is something the system cannot alter. The kind of check limits what the finished system can claim.

**Example: the three conditions in an economics task**

**Scaling Reproducibility**[12] re-runs the replication packages of published studies. *Connecting step:* working out how a package is meant to run and matching its output to the coefficients in the paper's tables. *Tools:* R, Stata and Python already execute the analyses. *Check:* the published coefficients, which are ground truth for reproduction, though not for whether the study's conclusions are right. Of the 254 executable papers, 82% finished without calling the system's adaptive coordinator. That path still uses model calls to extract metadata and coefficients, so the figure is about adaptive coordination, not all model use[12, pp. 7, 24].

**Example: a strong check and a weak one**

For its generation and tuning tasks, the **Pufibara** benchmark[22] tests its private behavioral evaluator on known-valid answers and deliberately off-target submissions. Its 132 repair tasks, of 232, are checked on interface, model checking and simulation, without a separate behavioral check. The agent cannot reach the evaluator while revising, or use its verdict for another attempt[22, pp. 9–10]. **Catalyst-Agent**[9] screens catalysts against predicted adsorption energies; when three selected materials were rechecked with a physics calculation, one fell outside the target window[9, p. 32]. Both systems loop successfully, but they support different claims. Agreement is not enough either: in **GAMESS modernization**[18], matching outputs could share the same build error[18, p. 8].

**Done when**

You can name the connecting step, the operations on either side of it, and a check you have tried on known cases; and you have written down what is known about the simpler options. Passing makes a small model-based baseline worth testing. It does not show that a language model, or an agent, is necessary.

# 4 Second question: does the model need to choose the next step?

Passing the first question makes a model somewhere in the loop worth trying. It does not justify an agent. The published systems give less to go on here: the cases motivate the questions below, but they do not validate them as predictive signals or show what letting the model choose is worth [inferred].

**B1. Results create decisions that a fixed procedure does not resolve.** In the same work samples, mark decisions that could not have been made from the original request: looking at a different artifact, asking for a new diagnostic, writing a test, changing approach, stopping, escalating. Then write the fixed procedure, with its branches, retries and escalation. If it plausibly covers the workload, build and test it first. No branch may hide an agent: "diagnose and repair" is not a step unless the procedure says how. Runs that differ from one another, or that retry, are not evidence on their own, because a fixed workflow branches and retries too.

**B2. An ordinary search or diagnostic routine is not already enough.** Say what the next decision requires someone to interpret, then compare it with the search, optimization and diagnostic methods available. "Which parameter value should be tried next?" is a reason to test an optimizer first. It does not rule a model out, and "which experiment would explain this failure?" does not rule one in, since some investigations follow a fixed routine.

**B3. Handling the exceptions is worth more than it costs.** Estimate how often the non-routine path occurs, what an expert spends on it, what a miss costs, and what the agent adds in running cost and oversight. Estimate running cost from tokens measured per run on sample work, the provider's prices and the expected volume, and allow for the variation between runs of the same task; Phase 6 of the design process sets out the check[D]. Rare cases with large consequences can matter more than frequent minor ones.

**Example: where choosing the next step shows up**

**SubDGuide**[19] rebuilds a smooth surface from a dense mesh. After a first attempt, its controller may ask for further rendered views before choosing to repair, roll back or stop[19, p. 5]. In **GAMESS modernization**[18], an agent diagnosing a numerical mismatch wrote its own bisection harness, a step nobody had scripted[18, p. 8]. **Pufibara**'s agent must "decide whether to collect further evidence, revise the candidate, or submit it"[22, p. 4].

**Example: where simpler control remains a credible alternative**

**Scaling Reproducibility**[12] calls its adaptive coordinator when a validation gate fails. Failure patterns that recur are written into deterministic rules or routing branches, through reviewed updates between versions of the pipeline[12, pp. 7, 9–10]. **Catalyst-Agent**[9] concedes that one of its showcase decisions "could be interpreted as exhaustive search over a binary parameter"[9, p. 23]. Its ablations still favor the agent's choices over random ones: 41% success on the oxygen reduction reaction, against 25.0% with random candidates, 23.3% with random modifications and 13.3% with both[9, pp. 29–30]. They do not compare the agent with a strong fixed policy under a matched budget. **SubDGuide**'s refinement stage improves both the output-to-target and the target-to-output P95 distances in 6 of 20 cases and leaves 14 outputs unchanged. It keeps an earlier verified state unless a proposal passes verification and improves the comparison score, which prevents a worse result under that score, not under every measure of quality[19, pp. 5, 7]. **TDFlow** reports the simpler Agentless system at $0.53 per issue and 61.0% success, against its own $1.51 and 88.8%[8, pp. 5, 11]; the two systems are equipped differently, so this is not a measure of what choosing the next step is worth.

**B4. Settle it with a matched comparison.** B1–B3 say where a comparison is worth running. Only the comparison justifies choosing an agent, and it is the Phase 7 baseline test applied to control:

- the same model, starting information, tools and check in both arms;
- comparable limits on calls, time and cost;
- a fixed workflow that is allowed realistic branches, retries and diagnostic calls ("fixed" must not mean deprived of feedback);
- cases that were not used to design either arm.

The only difference between the arms is who selects the next step. None of the 22 papers runs this comparison cleanly (Section 7), which is the main reason the second question cannot be answered from the literature alone.

**Done when**

You have written the strongest fixed procedure you can, listed the observed decisions it leaves unresolved, and recorded what evidence would show whether those gaps justify an agent.

# 5 Recap: the whole screen, and where it sits in the design process

Figure 2 puts the pieces in order. The screen starts from the Phase 1 brief of the design process[D], its first three decisions produce the choice that Phase 2 records, and its last step is the comparison that Phase 7 runs.

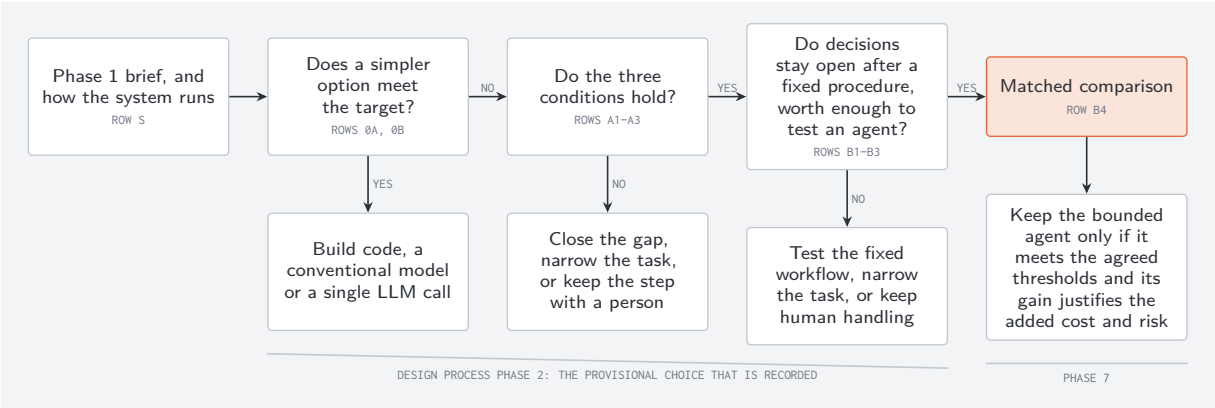



Figure 2: The screening decisions and the action each leads to. Row labels refer to the worksheet in Section 6. Phase 2 of the design process[D] records a provisional choice; a bounded-agent candidate then goes through Phases 3 to 6 before the matched comparison in Phase 7. The lower boxes are actions, not a one-to-one rendering of the four Phase 2 options. Where an answer is unknown, run the cheapest test that would settle it. The arrangement is my synthesis [inferred].

The worksheet's answers are also reused later in the design process, so they are written once:

| Worksheet rows | Design-process phase that uses them | As |
|---|---|---|
| S | Phase 2, scope; Phase 5, design tools, state and context | What starts a job, what notices that work is needed, and what state persists |
| 0a, 0b | Phase 2, the alternatives before the ladder and its first two options | Evidence that a simpler option meets or misses the target, or the test needed to decide |
| A1 | Phase 3, decompose and assign ownership | The step the model would own, and the steps that stay with code or a person |
| A2 | Phase 5, design tools, state and context | The tool list, access limits and permitted actions |
| A3 | Phase 4, define the evaluation | The check, its kind, and the known-good and known-bad cases |
| B1–B3 | Phase 2, leaving the fixed workflow | The fixed policy as specified, evidence about what it covers, the decisions it leaves unresolved and their expected value |
| B4 | Phase 7, evaluate, diagnose and iterate | The baseline comparison |

The appendix of the design process[D] works three examples through this worksheet: a phone agent that acts during a call, an investigation agent that resolves conflicting evidence, and a planning run that adapts from performance data.

# 6 Screening worksheet

**Comparing candidate tasks.** When several tasks compete, start from business problems rather than from the technology, and confirm that each activity happens often enough to justify the investment[29, "AI use case identification"]. Score the candidates on value and effort, rank them[28, pp. 25–28; 29, "Prioritize AI use cases"], and re-score periodically, since effort falls as capabilities change[28, p. 28]. Neither source gives validated weights; treat the ranking as a way to choose which candidate to screen first, not as the screen [inferred].

One page per candidate task. Carry forward the Phase 1 success measure and the limits on cost, delay and failure. Answer from work samples and existing records, and mark each answer as observed, estimated or unknown. Three to five samples are a first look, not a count of how often things happen.

| | Question | Write down |
|---|---|---|
| S | Does a request start the work, or must the system notice it? (Section 2) | On demand or always on; if always on, the bounded job an event starts, and what does the noticing |
| 0a | Would a script, a conventional model or an optimizer meet the target? | The strongest such option; evidence that it meets or misses the target, or "unknown" |
| 0b | Would one model call with the right context meet it? | Evidence on one call's accuracy and reliability, or the smallest test that would show it |
| A1 | Where does an expert interpret one result to decide the next input? | The step, marked on 3–5 real work samples |
| A2 | Which operations can run under your constraints, and which stay with a person? | Tools and how they are reached; access, cost, delay; human handoffs; permitted actions |
| A3 | What feedback is available during a run, and what is the final check? | For each: reference, proxy or judgment; coverage, cost, delay, owner; outcome on known-good and known-bad cases |
| B1 | Which decisions does the best fixed procedure leave unresolved? | The procedure you wrote, and the decisions it does not cover |
| B2 | Would an ordinary search or diagnostic routine make the next decision? | What the decision requires someone to interpret; the conventional method it was compared with |
| B3 | Is handling the exceptions worth more than it costs? | Frequency from broader records where available; expert time, cost of a miss, added oversight cost; running cost from measured tokens per run, prices, expected volume and run-to-run variation |
| B4 | What is the best fixed workflow, and how will it be compared? | Both arms, shared limits, untouched test cases |

Reading the result [inferred]:

- Evidence supports 0a or 0b: build the simplest option that works and verify it against the Phase 1 success

measure.

- Access or checking is inadequate (A2, A3): close the gap, narrow the task, or leave that step with a person.
- A fully written fixed procedure plausibly covers the workload: build and test it first. Coverage of the first few samples is provisional.
- Unresolved decisions remain and are worth enough (B1–B3): compare a bounded agent with the strongest fixed workflow (B4).
- Where an answer is unknown, write "unknown" and run the cheapest check that would settle it.

# 7 Limits, and where this attaches to the design process

**Limits.**

- *Selection.* The 22 are systems reported in papers and preprints, chosen by me from one group's publication list and from title and abstract searches. They are not a sample of attempted projects and include no failures that went unpublished.
- *No clean test of the second question.* None of the 22 holds tools, information and budget constant while changing only who selects the next step. SubDGuide comes closest: its rule-based controller shares the evidence and repair tools but also changes initial planning, and its one-shot variant has no feedback stage at all[19, pp. 6–7]. TDFlow compares complete systems with different tools and costs. Gains reported elsewhere could come from better tools, more information, verification or human specification.
- *Reading depth.* The classifications in the appendix come from text extractions of the full papers, in which some tables did not extract cleanly. One figure was checked against its PDF. Other figures, code repositories, execution traces and separately hosted supplements were not inspected, and no experiment was rerun.
- *Not validated.* The screen has not been applied prospectively. Treat a pass as a reason to run the comparison, not as a prediction of its result.

**Attachment to the design process (v1.10)[D].** Use the worksheet after the Phase 1 gate, as the input to Phase 2; Section 5 shows which phase reuses each row. Three parts of Phase 2 of the design process come from this document:

1. Before the ladder, a check for alternatives that use no language model (conventional models, optimizers). These are not a rung, since they can also be tools at any rung.
2. Before authorizing a bounded agent, three questions: what observation guides the next decision, what independently checks success, and how that check has been tested.
3. A tighter exit from the fixed-workflow rung. "Steps cannot be predicted" is not sufficient, because fixed workflows branch; the strongest maintainable fixed policy and the decisions it leaves unresolved must be written down.

# Authorship and use of AI tools

I am the sole author. I used AI tools in preparing this document and disclose their role here.

- *Claude* (Anthropic, through Claude Code) searched for and read the papers (and, for version 0.9.3, the guides on comparing candidate tasks), and drafted and revised the text, the table and the figure under my direction.
- *Codex* (OpenAI) acted as a second reader. It checked the papers, quotations, numbers and citations, and some sentences follow its suggested corrections closely.
- The distinction in Section 2 started from a conversation I had with *Gemini* (Google). The sources it cited were then checked against the originals.

Words taken from a source are in quotation marks with the source and page. Ideas taken from a source are cited to it. Statements marked [inferred] are my own synthesis. I set the questions, decided what to include and am responsible for the content.

# A The 22 systems

Table 1 lists the systems used as examples. Each reports an applied system in which a language model works with domain tools on an expert's task; papers mainly about evaluation were set aside. This is a purposive selection, not a systematic review. The last column uses the four options from Phase 2 of the design process: *code*; a *single model call*; a *fixed workflow*, where the path is set in advance, including programmed branches and retries; and an *agent*, where the model chooses the next step from results and decides when it is done. These assignments are my reading of each paper's methods section, not the authors' label. Several systems fix the outer procedure and keep model-directed components inside it, so a cell describes the level it names and not the whole system.

Table 1: The 22 systems: the connecting step automated, what checks the result, and the control structure as built. The control labels and the wording of each cell are mine [inferred].

| # | System | Connecting step | Check | Control as built |
|---|---|---|---|---|
| *One research group (engineering, materials, chemistry)* | | | | |
| 1 | Material Database Agent | Reading papers, tables and charts into database rows | Human-built databases, at evaluation only | Four prescribed stages; model-directed delegation and tool-use sessions inside them (agent inside a fixed workflow) |
| 2 | RocketSmith | Iterating between flight simulation, CAD and slicer | Simulated stability, slicer mass, four physical flights | Agent tool use inside a prescribed skill sequence; heavy human involvement |
| 3 | Pepti-Agent | Choosing the next sequence edit from predictor output | Three learned property classifiers (proxy) | Agent: a single loop with a turn cap |
| 4 | AgentsCAD | Choosing a geometry repair for a printing defect | Deterministic overhang recheck and solid validity; a vision-model verifier | Fixed outer workflow: its agents run "in a fixed phase order"[4, p. 10], with a tool-using design reasoner inside; the verifier-to-repair loop is not specified enough to classify |
| 5 | SLS process optimization | Choosing process parameters from past builds | Tensile and flexural tests on printed parts | Agent within a session; people start each build round |
| 6 | AgentD | Chaining retrieval, molecule generation and property prediction | Predicted properties and rule filters (proxy) | Fixed workflow in the reported end-to-end run |
| 7 | MatSciAgent | Turning a request into database queries or simulations | Tool outputs and repeat-run consistency; little independent validation of correctness | Router plus a per-task model loop (agent) |
| 8 | TDFlow | Locating a defect and writing the patch | Reproduction and regression tests | Fixed outer workflow with bounded agents inside |
| 9 | Catalyst-Agent | Choosing candidate materials and surface modifications | Predicted adsorption energies (proxy); three DFT checks | Agent at the screening level |

| # | System | Connecting step | Check | Control as built |
|---|---|---|---|---|
| *Other groups* | | | | |
| 10 | Symbolic discovery for fundamental analysis | Proposing valuation equations | Forecast error on held-out quarters | Fixed workflow: a propose–fit–evaluate loop |
| 11 | Deploy-Master | Inferring a build environment from documentation | Container build and a minimal run command | Fixed outer pipeline; control inside its refinement step not specified enough to classify |
| 12 | Scaling Reproducibility | Running replication packages and matching coefficients | Published coefficients | Fixed outer workflow; an agent diagnoses and repairs on failure |
| 13 | HLER | Proposing feasible questions and analyses for a dataset | Two researchers on question feasibility, at evaluation; a model reviewer on drafts; a person approves the final output | Fixed stage sequence with human gates; a model reviewer can request further analysis and another revision round |
| 14 | Causal inference workflow | Turning a causal plan into estimators and diagnostics | Known effects in synthetic benchmarks; diagnostics | Fixed notebook workflow; model-directed control inside it not established |
| 15 | Schwartz, Andrews & Shapiro | Reproducing, speeding up and extending published calculations | Published values, reruns, numerical references; adversarial model judgments for extensions | Fixed outer workflow |
| 16 | FinNextAssist | Gathering and reconciling financial evidence | Reference answers; rubric and model-judge grading | Fixed stages with an agent reasoning loop inside |
| 17 | CAi Copilot | Choosing molecular generation, filtering and evaluation steps | Rule checks; predicted properties; a blinded model judge | Agent |
| 18 | GAMESS modernization | Rewriting legacy Fortran and diagnosing breakage | Compilation, reference outputs, regression tests | Fixed per-file protocol with agent work inside |
| 19 | SubDGuide | Planning a control cage, then inspecting and repairing it | Geometric distance to the input mesh; topology gates | Single model call for the plan, then agent refinement |
| 20 | Quantum research system | Formalizing a protocol and implementing its calculations | Fixed tests and classical reference calculations | Project graph built progressively, with agent modules and human approval gates; protocol and tests locked before implementation |
| 21 | Oncology feature extraction | Mapping clinical documents to a structured schema | Clinician-reviewed values on an annotated subset | Fixed workflow: set routing over code steps and single model calls |
| 22 | Pufibara (Modelica) | Deciding what simulation evidence or repair to pursue | Private final evaluator: interface, model checking and simulation for repair tasks; behavioral checks as well for generation and tuning | Agent |

# Sources

Bracketed numbers in the text refer to these entries; page numbers are PDF page positions. arXiv identifiers and DOIs are links.


[1] Material Database Agent. arXiv:2605.04278.

[2] RocketSmith. arXiv:2606.00097.

[3] Pepti-Agent. arXiv:2606.15422.

[4] AgentsCAD. arXiv:2607.02448.

[5] AI agentic selective laser sintering process optimization. arXiv:2608.25928.

[6] AgentD: LLM agent for modular task execution in drug discovery. *J. Chem. Inf. Model.* 66(4), 2026; arXiv:2507.02925.

[7] MatSciAgent. *Communications Materials* 7, 131 (2026). doi:10.1038/s43246-025-00994-x.

[8] TDFlow: Agentic Workflows for Test Driven Development. EACL 2026; arXiv:2510.23761.

[9] Catalyst-Agent. arXiv:2603.01311.

[10] Self-Evolving Multi-Agent Symbolic Discovery for Financial Fundamental Analysis. arXiv:2609.32746.

[11] Deploy-Master. arXiv:2601.03513.

[12] Xu and Yang, Scaling Reproducibility: An AI-Assisted Workflow for Large-Scale Replication and Reanalysis. arXiv:2602.16733.

[13] HLER: Human-in-the-Loop Economic Research via Multi-Agent Pipelines. arXiv:2603.07444.

[14] A Human-Augmenting Agentic Workflow for Observational Causal Inference. arXiv:2607.22443.

[15] Schwartz, Andrews and Shapiro, An LLM Workflow That Reproduces, Improves, and Extends Published Economics Research. Working paper, September 2026. `https://economics.mit.edu/sites/default/files/2026-09/metascience.pdf`

[16] FinNextAssist. arXiv:2610.03174.

[17] CAi Copilot. arXiv:2608.06961.

[18] An Agentic Workflow for Legacy HPC Modernization (GAMESS). arXiv:2608.12249.

[19] SubDGuide. arXiv:2610.11721.

[20] Demonstration of a Structured Agentic Workflow for Applied Quantum Computing Research. arXiv:2610.05304.

[21] From Analytics to Tumor Boards: An Evidence-Linked Multi-Agent Workflow for Oncology Feature Extraction. arXiv:2608.28974.

[22] Pufibara Agent Harness and the Modelica Agent Workflow Benchmark. arXiv:2608.23653.


### The other two documents of this set.


[D] J. A. Arroyo, *Agentic System Design Process* (v1.10, November 2026). File `design_process/design_process.pdf`.

[G] J. A. Arroyo, *Learning to Build Agentic Systems: An instructional compilation guide built from public university courses* (October 2026). File `guide/agentic_guide.pdf`.


**Background for Section 2.** These are cited only for the specific points attributed to them there.


[23] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020. Chapter 2, §2.3. `https://aima.cs.berkeley.edu/`

[24] M. Wooldridge and N. R. Jennings, Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review* 10(2), 115–152, 1995. doi:10.1017/S0269888900008122.

[25] Y. Lu et al., Proactive Agent: Shifting LLM Agents from Reactive Responses to Active Assistance. ICLR 2025; arXiv:2410.12361.

[26] T. R. Sumers, S. Yao, K. Narasimhan and T. L. Griffiths, Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*, 2024; arXiv:2309.02427.

[27] R. Parasuraman, T. B. Sheridan and C. D. Wickens, A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A* 30(3), 286–297, 2000. doi:10.1109/3468.844354.


**Background for comparing candidate tasks (Section 6).** Checked on 2 November 2026.


[28] OpenAI, Identifying and scaling AI use cases. cdn.openai.com (PDF).

[29] Microsoft Learn, Cloud Adoption Framework, Plan for AI adoption. `https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai/plan`